SDG 1 - No Poverty

Machine Learning Based Poverty Prediction

```
from google.colab import files

uploaded = files.upload()
```

```
Choose Files poverty_dataset.csv
poverty_dataset.csv(text/csv) - 2878 bytes, last modified: 7/20/2026 - 100% done
Saving poverty_dataset.csv to poverty_dataset (1).csv
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
df = pd.read_csv("poverty_dataset.csv")

print("First 5 Records")
display(df.head())

print("\nDataset Shape:", df.shape)

print("\nDataset Information")
df.info()

print("\nStatistical Summary")
display(df.describe())
```

First 5 Records

	Literacy_Rate	Employment_Rate	Household_Income	Healthcare_Access	Population_Density	Electricity_Access	Internet_Access
0	83	94	132235	11	1616	79	73
1	96	94	89163	6	3246	87	79
2	73	94	45939	13	512	87	27
3	59	79	68925	3	828	94	72
4	87	75	62941	4	3448	57	79

Dataset Shape: (100, 8)

Dataset Information

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 8 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   Literacy_Rate       100 non-null    int64
 1   Employment_Rate     100 non-null    int64
 2   Household_Income    100 non-null    int64
 3   Healthcare_Access   100 non-null    int64
 4   Population_Density  100 non-null    int64
 5   Electricity_Access  100 non-null    int64
 6   Internet_Access     100 non-null    int64
 7   Poverty             100 non-null    int64
dtypes: int64(8)
memory usage: 6.4 KB
```

Statistical Summary

	Literacy_Rate	Employment_Rate	Household_Income	Healthcare_Access	Population_Density	Electricity_Access	Internet_Acc
count	100.000000	100.00000	100.000000	100.000000	100.000000	100.000000	100.00
mean	71.170000	64.27000	87909.960000	6.670000	2548.940000	77.110000	57.36
std	15.592056	17.47544	37159.550404	4.022801	1323.126904	13.522816	23.26
min	46.000000	35.00000	20854.000000	1.000000	243.000000	50.000000	20.00
25%	58.750000	48.75000	56559.250000	3.000000	1336.500000	69.750000	35.00
50%	69.000000	66.00000	86520.500000	6.500000	2621.500000	78.000000	56.00
75%	84.500000	77.25000	118200.000000	9.250000	3595.000000	87.250000	77.50
max	97.000000	95.00000	148376.000000	14.000000	4949.000000	99.000000	99.00

```
print("Missing Values")
print(df.isnull().sum())

print("\nDuplicate Rows:", df.duplicated().sum())

# Remove duplicates
df = df.drop_duplicates()

print("\nDataset Shape After Cleaning:", df.shape)
```

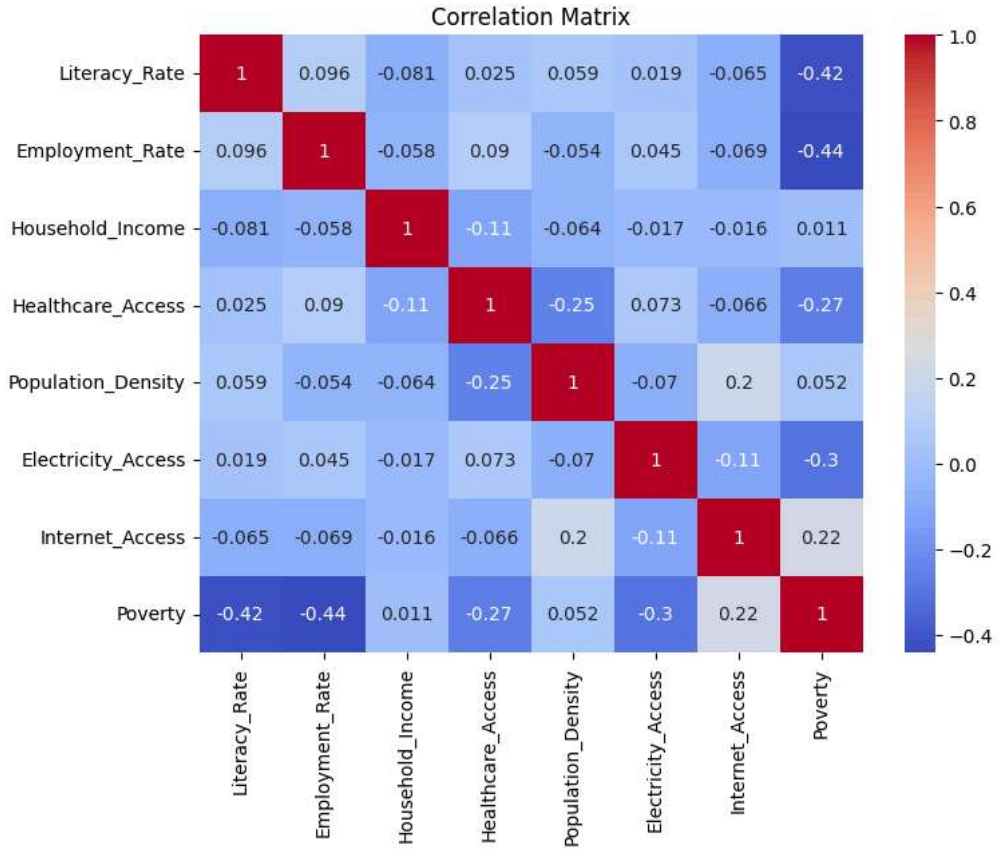
```
Missing Values
Literacy_Rate         0
Employment_Rate       0
Household_Income      0
Healthcare_Access     0
Population_Density    0
Electricity_Access    0
Internet_Access       0
Poverty               0
dtype: int64

Duplicate Rows: 0

Dataset Shape After Cleaning: (100, 8)
```

```
print("Average Household Income:", np.mean(df["Household_Income"]))

print("Average Literacy Rate:", np.mean(df["Literacy_Rate"]))

print("Average Employment Rate:", np.mean(df["Employment_Rate"]))
```

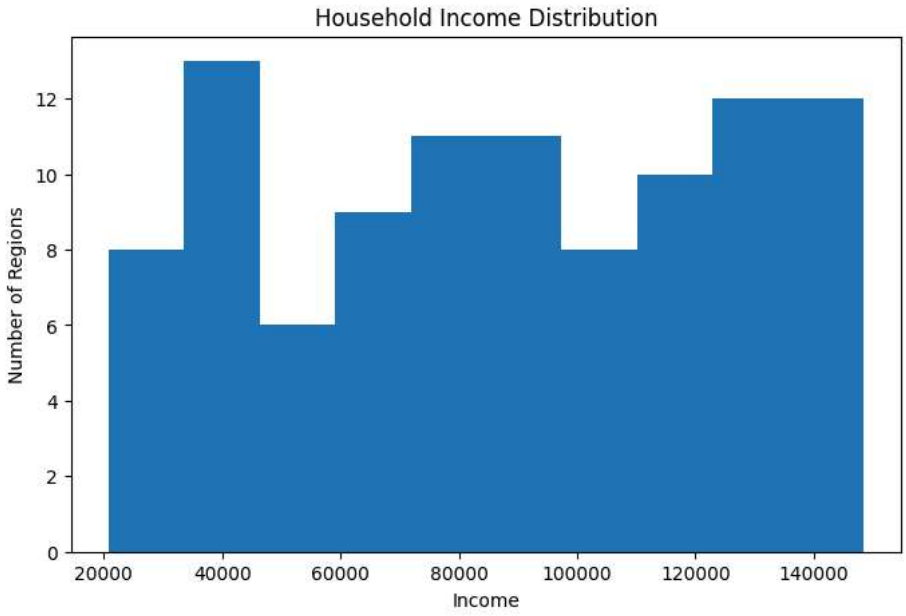
```
poverty_percentage = np.mean(df["Poverty"]) * 100

print("Poverty Percentage:", round(poverty_percentage,2),"%")
```

```
Average Household Income: 87909.96
Average Literacy Rate: 71.17
Average Employment Rate: 64.27
Poverty Percentage: 23.0 %
```

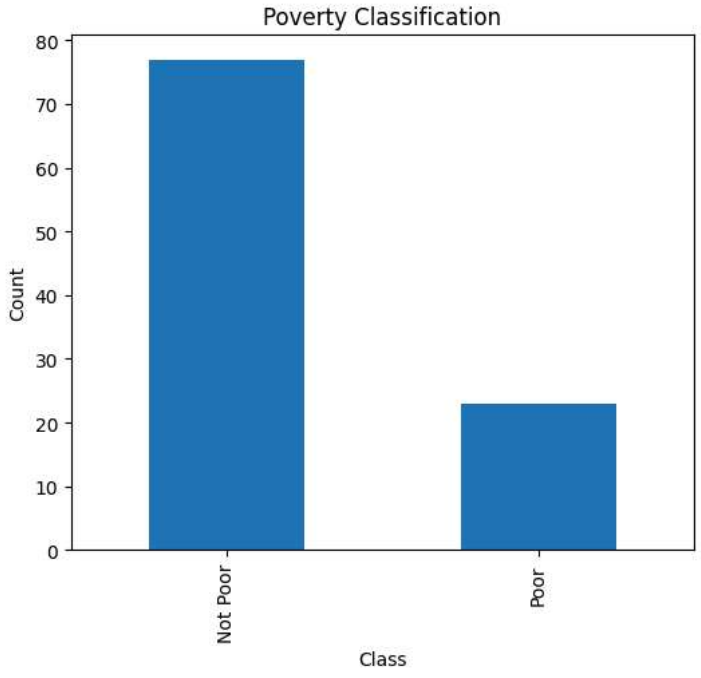
```
plt.figure(figsize=(8,6))

sns.heatmap(df.corr(), annot=True, cmap="coolwarm")

plt.title("Correlation Matrix")

plt.show()
```




```
plt.figure(figsize=(8,5))

plt.hist(df["Household_Income"], bins=10)

plt.title("Household Income Distribution")

plt.xlabel("Income")

plt.ylabel("Number of Regions")

plt.show()
```




```
plt.figure(figsize=(6,5))

df["Poverty"].value_counts().plot(kind="bar")

plt.title("Poverty Classification")

plt.xlabel("Class")

plt.ylabel("Count")

plt.xticks([0,1],["Not Poor","Poor"])

plt.show()
```




```
X = df.drop("Poverty", axis=1)

y = df["Poverty"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
```

```
    test_size=0.2,
    random_state=42
)
```

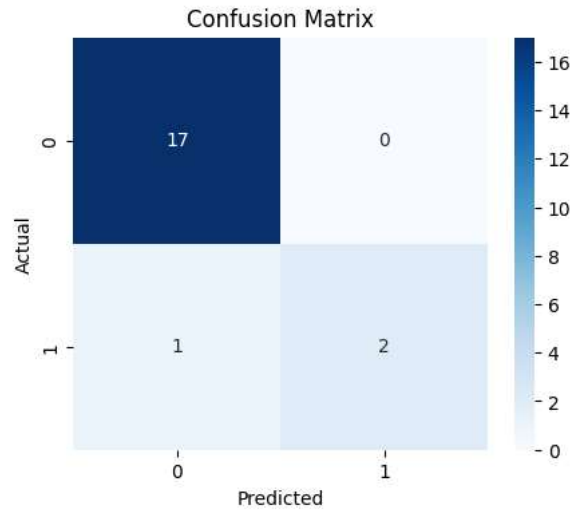
```
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

model.fit(X_train, y_train)
```

```
RandomForestClassifier(random_state=42)
```

```
prediction = model.predict(X_test)

accuracy = accuracy_score(y_test, prediction)

print("Accuracy:", round(accuracy*100,2),"%")
```

```
Accuracy: 95.0 %
```

```
cm = confusion_matrix(y_test, prediction)

plt.figure(figsize=(5,4))

sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")

plt.title("Confusion Matrix")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()
```




```
print(classification_report(y_test, prediction))
```

```
              precision    recall  f1-score   support

           0       0.94      1.00      0.97        17
           1       1.00      0.67      0.80         3

    accuracy                           0.95        20
   macro avg       0.97      0.83      0.89        20
weighted avg       0.95      0.95      0.95        20
```

```
importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
})
```

```
importance = importance.sort_values(
    by="Importance",
    ascending=False
)

display(importance)
```

	Feature	Importance
0	Literacy_Rate	0.241705
1	Employment_Rate	0.233153
5	Electricity_Access	0.135195
6	Internet_Access	0.115113
4	Population_Density	0.105138
3	Healthcare_Access	0.099258
2	Household_Income	0.070437

Next steps: Generate code with `importance` New interactive sheet

```
plt.figure(figsize=(8,5))

plt.bar(
    importance["Feature"],
    importance["Importance"]
)

plt.xticks(rotation=45)

plt.title("Feature Importance")

plt.show()
```